

ER2 ASTRE

Master 2 Informatique Fév 2024

UE 5IN455

Année 2024-2025

2 heures – **Documents manuscrits autorisés**
Tout appareil de communication électronique interdit (téléphones...)

Le sujet est composé de sections indépendantes que l'on pourra traiter dans l'ordre souhaité. On vous demande de rédiger chaque partie sur une copie séparée.

Partie 1 : Systèmes temps réel

1 Ordonnancement temps réel pour processeur monocœur (5 points)

Considérons l'ensemble suivant de tâches indépendantes, synchrones et à priorité fixe. Il s'exécute sur un processeur monocœur. Les tâches ont des échéances contraintes ($D \leq T$). *Nous ne faisons aucune hypothèse*

Tâche	Budget	Période	Echéance
T0	1	10	8
T1	3	14	9
T2	2	15	8
T3	3	21	20
T4	3	35	4

Question 1. (0.5 point) Condition nécessaire [cours]

Rappelez la condition nécessaire d'ordonnabilité pour les processeurs monocœurs. Est-elle satisfaite pour cet ensemble de tâches ? Notez que $2 \times 3 \times 5 \times 7 = 210$.

Le taux d'utilisation doit être inférieur à 1.

$$U = \frac{1}{10} + \frac{3}{14} + \frac{2}{15} + \frac{3}{21} + \frac{3}{35} = \frac{142}{210} \leq 1.$$

Potentiellement, le système de tâches peut être ordonnancé sur un processeur monocœur.

Question 2. (1 point) Pire Temps de Réponse [cours]

Rappelez et expliquez la formule de calcul du pire temps de réponse. Rappelez également les notations.

$$R_i^n = \sum_{j \in hp(i)} \lceil \frac{R_i^{n-1}}{T_j} \rceil \times C_j \leq D_i$$

Nous prenons en compte les temps d'exécution des tâches ayant une priorité plus élevée que celle de la tâche i , dont nous voulons calculer le pire temps de réponse de manière itérative. $hp(i)$ désigne l'ensemble des tâches ayant une priorité supérieure ou égale à celle de la tâche i .

Question 3. (1.5 points) Optimal Priority Assignment [cours]

Décrivez l'algorithme OPA (Optimal Priority Assignment) permettant de calculer toutes les affectations de priorités rendant cet ensemble de tâches ordonnançable. Veuillez à détailler toutes les propriétés à vérifier.

Nous recherchons toutes les tâches auxquelles nous pouvons attribuer la priorité la plus basse en calculant leurs pires temps de réponse et en vérifiant qu'elles respectent leurs échéances. Ensuite, pour chacune de ces tâches t_i , nous définissons un nouvel ensemble de tâches réduit, dont t_i a été retirée, et nous appliquons la même approche, c'est-à-dire que nous trouvons toutes les tâches auxquelles nous pouvons attribuer la priorité la plus basse. De cette manière, nous déterminons toutes les affectations de priorités possibles rendant cet ensemble de tâches ordonnançable.

Question 4. (2 points) Affectations de Priorités Valides [cours]

En utilisant les questions précédentes, trouvez les assignations de priorité qui rendent ce système de tâches ordonnançable. Expliquez votre raisonnement.

Nous pouvons explorer systématiquement les cinq possibilités pour attribuer la priorité la plus basse, puis répéter le processus avec les sous-ensembles de tâches de priorités plus élevées. Cependant, ici, nous déduisons le résultat final grâce à des considérations de base.

La charge de travail à $t = 0$ (lorsque toutes les tâches démarrent de manière synchrone) est $1 + 3 + 2 + 3 + 3 = 13$. Seule T3 peut avoir la priorité la plus basse (échéance supérieure à 13). Nous vérifions cela en calculant le temps de réponse de T3, et son échéance est respectée.

Une fois T3 retirée de l'ensemble des tâches, la charge de travail à $t = 0$ est $1 + 3 + 2 + 3 = 9$. Ainsi, seule T1 peut avoir la priorité la plus basse (échéance supérieure à 9). Nous vérifions cela en calculant le temps de réponse de T1, et son échéance est respectée.

Une fois T1 également retirée de l'ensemble des tâches, la charge de travail à $t = 0$ est $1 + 2 + 3 = 6$. Par conséquent, T0 et T2 peuvent avoir la priorité la plus basse (échéance supérieure à 6). Notez que les tâches T0, T2 et T4 ne sont pas activées deux fois pendant cette période. T4 ne peut pas avoir la priorité la plus basse (son échéance est inférieure à 6). Si T2 a la priorité la plus basse, T0 et T4 peuvent avoir la priorité la plus élevée. Si T0 a la priorité la plus basse parmi les trois tâches restantes, T4 doit avoir la priorité la plus élevée car la charge de travail de T4 et T2 est $3 + 2 = 5$.

Tâche	Budget	Période	Echéance	Temps de réponse
T4	3	35	4	3
T2	2	15	8	5
T0	1	10	8	6
T1	3	14	9	9
T3	3	21	20	12

Tâche	Budget	Période	Echéance	Temps de réponse
T4	3	35	4	3
T0	1	10	8	4
T2	2	15	8	6
T1	3	14	9	9
T3	3	21	20	13

Tâche	Budget	Période	Echéance	Temps de réponse
T4	3	35	4	3
T0	1	10	8	4
T2	2	15	8	6
T1	3	14	9	9
T3	3	21	20	12

Notez que ces affectations correspondent à celles que DMS (Deadline Monotonic Scheduling) donnerait, T0 et T2 ayant le même délai. Cependant, il n'était pas possible de conclure directement. En effet, la condition $\sum \frac{C_i}{D_i} \leq n(2^{\frac{1}{n}} - 1)$ n'est pas satisfaite ($\frac{11}{12} \not\leq 5(2^{\frac{1}{5}} - 1) \approx 0.74$), et cette condition est seulement suffisante.

2 Couverture de code (2 points)

Question 1. (1 point) Définitions [cours]

Indiquer ce que doit faire un banc de tests pour garantir la couverture de code selon les approches 1) Statement Coverage, 2) Decision Coverage et 3) Modified Condition/Decision Coverage. Une attention particulière sera accordée à la lisibilité et la clarté de la réponse.

Voir cours.

Question 2. (1 point) Applications de MC/DC

On considère le code ci-dessous. Illustrez l'approche MC/DC en fournissant les valeurs des conditions à utiliser dans les tests de la couverture pour satisfaire l'approche MC/DC. Une attention particulière sera accordée à la lisibilité et la clarté de la réponse. Vous pouvez renommer les conditions par X, Y et Z.

```
1 if ((a == 0) && ((b != 1) && (c == 2))) {F1(a, b, c)}; F2(a, b, c) ;
```

Trivial with only *and*.

X	Y	Z	D
T	T	T	T
F	T	T	F
T	F	T	F
T	T	F	F

3 Ordonnancement sur processeur multicœur (3 points)

L'ensemble suivant de tâches indépendantes, synchrones et à échéances implicites est exécuté sur un **processeur à deux cœurs** en suivant la politique d'ordonnancement Global Deadline Monotonic.

Tâche	Budget	Période
T0	1	2
T1	2	3
T2	2	4

Question 1. (1 point) Instant Critique [cours]

Rappeler pourquoi l'instant critique est important dans le calcul du temps de réponse et dans quelles circonstances il se produit sur un **processeur mono-cœur**.

cours

Question 2. (1 point) Ordonnancement Global Deadline Monotonic

Expliquer le fonctionnement d'un ordonnanceur Global Deadline Monotonic sur un **processeur multicœur** et donner le chronogramme de l'ensemble de tâches donné exécuté par un tel ordonnanceur sur un **processeur à deux cœurs**. Justifier l'intervalle de temps sur lequel le chronogramme se déroule.

cours. Intervalle de 12

Question 3. (1 point) Worst Case Response Time

Indiquer le temps de réponse dans le pire cas pour les différentes tâches et conclure sur ce que cet exemple met en évidence.

$R_0 = 1, R_1 = 2, R_2^1 = 3, R_2^2 = 4$. Il s'agit d'une anomalie d'ordonnancement, l'instant critique ne se produit pas lorsque les tâches sont toutes libérées au même moment.

Partie 2. Méthodes symboliques

DD vs SAT (5 points)

Question 1. (1 points) Expliquez quelles sont les entrées et les sorties d'un SAT solver. On sera aussi précis que possible, notamment sur le format des entrées.

En entrée une formule booléenne F de N variables booléennes, en CNF, c'est à dire sous la forme d'un produit (AND, une liste) de sommes (OR, les clauses). Techniquement le format est dimacs.

En sortie:

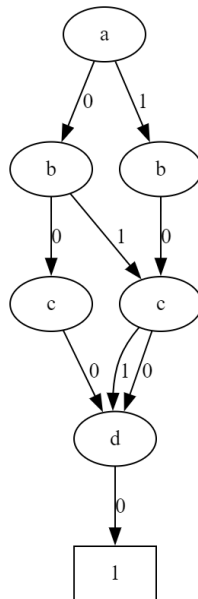
- SAT : et une valuation des variables v satisfaisant la formule : $F(v)=\text{true}$
- UNSAT : la formule n'est pas satisfaisable, et potentiellement une explication de pourquoi : un UNSAT-core. (Mais on n'a pas vraiment parlé d'unsat-core en cours)

Barème :

50 % entrée : une formule bool (20 %) en CNF (20%) + 10% pour explications, termes clause/littéral etc...

50 % sortie; 20 % SAT + un modèle v 10 % ; 20 % UNSAT + signification

Question 2. (2,5 points) On suppose le diagramme de décision suivant qui représente les états accessibles d'un système de quatre variables.



- A) Combien d'états sont représentés par ce diagramme ?
- B) Dessinez le DD représentant le sous ensemble de ces états satisfaisant "d=0 AND b=0".
- C) Donnez les principes d'un algorithme récursif permettant d'implanter le calcul demandé en B).
- D) Quelle est la complexité de cette opération en fonction de la taille du DD en entrée ?

A) Il y a 5 états : 0000, 0100, 0110, 1010, 1000

B) d vaut déjà zéro, donc il faut simplement couper/retirer l'arc du noeud b de gauche étiqueté 1 qui va vers le c de droite. La structure ne bouge pas (les noeuds b sont similaires mais n'ont pas les même descendants, il n'y plus que 3 états représentés 0000,1010,1000.

C) Il faut une fonction comme : BDD filter (BDD node, Expr e)

C'est à dire qui prend et rend un noeud, peu importe la forme précise de l'argument qui pose le critère. Par exemple :

```

BDD varEq (BDD node, string var, bool value) {
    if (node.name == var) {
        if (value)
            return BDD(node.name, 0 , node[1])
        else
            return BDD(node.name, node[0] , 0)
    } else {
        //recursion
        return BDD(node.name, varEq(node[0],var,value) , varEq(node[1],var,value));
    }
}

```

On pourrait obtenir le DD cible depuis l'entrée *in* par : `varEq( varEq ( in, "b", false), "d", false)` ou via un AND opération primitive entre le résultat des deux invocations.

D) Elle est proportionnelle au nombre de chemins, ce qui n'est pas bon. Il faut absolument munir nos opération sur le DD d'un cache, ce qui nous ramène en complexité proportionnelle au nombre de noeuds du DD.

Barème : A) 20 %

B) 30 %

C) 30 % dès qu'on arrive à proposer une signature qui prend et rend un noeud de l'arbre. C'est la clé pour interpréter correctement "algorithme récursif". L'opération est une variante sur AND/OR vu en cours.

D) 20 % dès qu'on arrive à poser une complexité cohérente avec la réponse en C. +20% bonus pour toute évocation d'un cache pour réduire la complexité.

Question 3. (1,5 points) Expliquez les principes d'une démarche de preuve d'un invariant d'un système (AGp où p est un prédicat sur les états) par induction à l'aide d'un solveur SAT/SMT. Quelles variables sont introduites et quelles sont les contraintes exprimées dessus, enfin comment interpréter la réponse du solveur ?

1. On définit des variables pour chaque variable du système. On en définit deux par variable du système; on va poser $s0, s1$ ces deux jeux de variables.

2. On définit une contrainte qui lie $s0$ à $s1$; c'est la relation de transition du système, donc on $assert(succ(s0, s1))$ si $succ$ désigne un lien de successeur immédiat.

3. On assert que $s0$ satisfait p , $assert(p(s0))$

4. On assert que $s1$ ne satisfait pas p , $assert((not p(s1)))$

Si UNSAT, mais que l'état initial du système satisfait p , on a prouvé par induction que AGp .

Sinon, inconclusif, l'état choisi pour $s0$ n'est peut-être pas accessible.

Barème :

30 % on pose deux jeux de variables $s0, s1$

20 % relation successeur

30 % $s0$ satisfait p , $s1$ ne satisfait pas

20 % interprétation seulement sur UNSAT (semi décision)

SMT (5 points)

2	7	6	→15
9	5	1	→15
4	3	8	→15
↙15	↓15	↓15	↓15
			↘15

En mathématiques, un carré magique d'ordre n est composé de n^2 entiers strictement positifs, écrits sous la forme d'un tableau carré. Ces nombres sont disposés de sorte que leurs sommes sur chaque rangée, sur chaque colonne et sur chaque diagonale principale soient égales. On nomme alors constante magique la valeur de ces sommes.

Un carré magique normal est un cas particulier de carré magique, constitué de tous les nombres entiers de 1 à n^2 , où n est l'ordre du carré.¹

Question 4. (2 points) Expliquez comment encoder le problème pour construire un carré magique normal d'ordre 3 à l'aide d'un solveur SMT. Pour éviter la syntaxe lourde des matrices, on demande à cette question d'utiliser 9 variables $A1, A2, A3, B1, \dots$ et de répondre en syntaxe SMT. On commencera par poser les "variables" et leur domaine, puis les contraintes.

¹Image et description issue de Wikipedia.

```

; Déclaration des variables représentant les cases du carré
(declare-const A1 Int)
(declare-const A2 Int)
(declare-const A3 Int)
(declare-const B1 Int)
(declare-const B2 Int)
(declare-const B3 Int)
(declare-const C1 Int)
(declare-const C2 Int)
(declare-const C3 Int)

; Déclaration de la variable Total (constante magique)
(declare-const Total Int)

; Contraintes de domaine : chaque case est dans [1, 9]
(assert (and (>= A1 1) (<= A1 9)))
(assert (and (>= A2 1) (<= A2 9)))
(assert (and (>= A3 1) (<= A3 9)))
(assert (and (>= B1 1) (<= B1 9)))
(assert (and (>= B2 1) (<= B2 9)))
(assert (and (>= B3 1) (<= B3 9)))
(assert (and (>= C1 1) (<= C1 9)))
(assert (and (>= C2 1) (<= C2 9)))
(assert (and (>= C3 1) (<= C3 9)))

; Contraintes d'unicité : toutes les cases contiennent des valeurs distinctes
(assert (distinct A1 A2 A3 B1 B2 B3 C1 C2 C3))

; Contraintes d'égalité des sommes (l'utilisation de la variable Total simplifie les contraintes)
(assert (= (+ A1 A2 A3) Total))
(assert (= (+ B1 B2 B3) Total))
(assert (= (+ C1 C2 C3) Total))
(assert (= (+ A1 B1 C1) Total))
(assert (= (+ A2 B2 C2) Total))
(assert (= (+ A3 B3 C3) Total))
(assert (= (+ A1 B2 C3) Total))
(assert (= (+ A3 B2 C1) Total))
(check-sat)
(get-model)

```

Barème :

20 % déclaration des variables (*9), declare-const ou fun sans paramètre, domaine Int.

25 % on contraint leur domaine aux entiers de $1..N^2$ (*9)

25 % on assert distinct, le carré est "normal"

30 % on exprime l'égalité des sommes (a priori avec une variable de plus, mais on tolère 15 en dur.)

Question 5. (2 points) On suppose au contraire un encodage qui utilise une matrice (*declare-const* M (*Array Int* (*Array Int Int*))) de façon à pouvoir faire varier l'ordre n . Expliquez les difficultés que cela pose pour la définition des contraintes. (On ne demande pas de coder une solution complète vu qu'il y a une difficulté majeure, mais plutôt d'expliquer les grandes lignes de l'encodage et de mettre en évidence la difficulté ; on accepte pour cette réponse aussi des réponses en syntaxe mathématique classique).

Contraintes :

I. Domaine des variables : **faisable**

$$\forall i, j \in N, 0 \leq i < N^2, 0 \leq j < N^2$$

$$\Rightarrow 1 \leq M[i, j] \leq N$$

II. Distinct : chiant mais **faisable** via une explosion quadratique : pas moyen de faire juste "distinct" comme dans la solution basique. Il faut en gros :

$$\forall i, j, i', j' \in N,$$

$$0 \leq i < N^2, 0 \leq j < N^2,$$

$$i < i' < N^2, j < j' < N^2$$

$$\Rightarrow M[i, j] \neq M[i', j']$$

Sommes : ouch, **pas faisable** en fait (du moins simplement, il y a une façon récursive mais c'est très cher et pas pratique du tout).

Bref, il n'existe pas de syntaxe qui supporte $\Sigma_i(M[i])$ pour construire une expression SMT, malheureusement. Les forall/exists qui sont disponibles ne permettent pas vraiment de construire une somme.

Barème:

20 % pas de souci sur les domaines, via des implications (si indice dans la plage, on contraint les valeurs du Array).

40 % distinct n'est pas paramétrique, il faut une quadratique pour l'exprimer

40 % on voit que la somme pose un gros souci.

Morale, il vaut mieux piloter de l'extérieur le solver (e.g. en python, générer le SMT correspondant à Q5) que d'utiliser des Array là où un bloc de variables ordinaires suffisent.

Question 6. (1 point) Peut-on encoder et résoudre ce problème (avec un ordre N fixé par énoncé) à l'aide d'un solver SAT ? *Justifiez* votre réponse en un ou deux paragraphes qui expliquent soit une ligne d'encodage candidat, soit pourquoi vous pensez que c'est impossible.

Encodage en SAT, avec un N fixé est bien entendu possible.

Variables : chacune est représentée par $\ln 2(N^2)$ variables bool, aussi appelés *bits*. e.g. codage à base 2 comme les int machine. Disons pour $N = 3$, il faut 4 bits par variable.

Domaines : on peut écrire une contrainte qui permet de comparer une variable v qui occupe un champ de bits $v3, v2, v1, v0$ à une constante qui a le même support e.g. $9 = 1001$. Donc ici on va avoir une assertion $v3 \vee v2 \vee v1 \vee v0$ (non nul) et une clause $(v3 \wedge v2 \wedge \bar{v1}) \vee v3$ pour inférieur ou égal à 9.

Distinct : c'est également faisable, on exige un bit de différence entre toute paire de champs de bits représentant les variables

Somme : pénible mais oui on sait coder la somme comme une fonction booléenne aussi ; $c=a+b$ donne en gros ($\oplus$ is xor):

$$c0 = a0 \oplus b0$$

$$c1 = (a0 \wedge b0) \vee (a1 \wedge \bar{b1}) \vee (\bar{a1} \wedge \bar{b1}) : a1 \oplus b1;$$

Barème : 50 % oui c'est possible

50 % un peu de justification